MILITARY INSTITUTE OF SCIENCE & TECHNOLOGY

Department of Computer Science and Engineering

TERM PAPER

CSE 307: Operating Systems (Part B)

Decision Tree Based Page Replacement Under Changing Memory Access Patterns

Submitted By

Fahim Fazle Aziz Bin Zahid

ID: 202414004

Level 3, Term 1, Section A

Submitted To

Lec Khaled Hasan Irfan

Lecturer

Department of Computer Science and Engineering

Military Institute of Science & Technology

GitHub Repository

github.com/Fahim5492/LearnedAlgoDecisionTree

Abstract

Page replacement is a core part of virtual memory management because the operating system must choose which resident page to remove when a page fault occurs and all available frames are occupied. This work studies whether a lightweight Decision Tree can use recent page-access behavior to make useful eviction decisions and how reliably those decisions hold under workload change.

The learned policy was trained on synthetic locality-heavy, sequential, random, and bursty workloads. Optimal replacement was used as an offline teacher, while the Decision Tree itself used only two historical features: recency and recent access frequency. The final evaluation deliberately shifted from locality-heavy to uniformly random page references.

Across ten evaluation traces, the learned policy achieved an average hit ratio of 0.6540 during locality-heavy access, compared with 0.5802 for LRU and 0.5389 for FIFO. After the shift, its hit ratio fell to 0.2019. Its agreement with the recorded Optimal victim also dropped from 50.50% to 24.93%, while average confidence increased from 58.88% to 77.54%. In this controlled experiment, historical access features improved replacement decisions while the workload contained useful structure, but their predictive value and the reliability of the model's confidence scores deteriorated sharply when that structure disappeared.

1. Problem Introduction

In a demand-paged virtual memory system, a page fault may require the operating system to remove one of the pages already in memory to make room for an incoming page. Tanenbaum and Bos describe this as the operating system having to choose a page to evict when a page fault occurs [1]. The choice is important because removing a heavily reused page may cause another page fault shortly afterward.

The theoretical Optimal replacement policy provides a useful reference point. It removes the page whose next use is furthest in the future. Tanenbaum and Bos summarize the limitation clearly: "The best possible page replacement algorithm is easy to describe but impossible to actually implement" [1, p. 209]. A real system does not know the future reference string, so practical replacement policies must depend on information already available.

This work treats page replacement as a small prediction problem. Rather than defining another fixed heuristic, a Decision Tree was trained to estimate which resident page most closely resembles the victim that Optimal would select. The learned policy therefore attempts to approximate a future-aware decision using historical information only.

The main interest of the experiment is not only whether the learned policy performs well, but whether its behavior remains reliable when the workload changes. A policy based on recent history implicitly assumes that historical patterns provide useful information about future page reuse. That assumption is reasonable under locality-heavy access but becomes weaker when page references are uniformly random.

The experiment therefore asks three questions:

- Can a lightweight learned replacement policy outperform FIFO and LRU when the workload has strong locality?
- What happens when the access pattern shifts from locality-heavy to uniformly random?
- Does the confidence reported by the learned policy remain meaningful after the shift?

2. Implementation Summary

The simulator implements FIFO, LRU, Optimal, and a learned Decision Tree policy. The first three provide comparison baselines, while the main implementation focus is the learned policy and its training process.

2.1 Training Workloads

The Decision Tree was trained using automatically generated page-reference traces rather than manually labeled examples. Four workload families were used: locality-heavy, sequential, uniformly random, and bursty.

An equal number of traces was generated for each workload family, providing balance at the trace-count level. The number of candidate examples produced by each family could still differ because different workload types produced different numbers of page faults. Within each category, parameters and reference sequences were varied using controlled random seeds.

The purpose was to give the model broad exposure to different access behaviors without specifically training it for the final workload transition. The model therefore encountered locality-heavy and random workloads during training, but the training procedure did not encode a fixed locality-to-random transition.

2.2 Training Labels

Optimal replacement acted as the offline teacher. Whenever a training trace caused a page fault while all frames were occupied, each resident page became an eviction candidate. The candidate selected by Optimal received the positive label 1, while the remaining candidates received 0.

This process generated approximately 75,340 labeled candidate examples. Since the labels were generated automatically from the same theoretical benchmark, the training data remained consistent and did not depend on manually selected victims.

2.3 Input Features

The learned policy uses two historical features for each resident page.

Recency is the number of reference steps since that page was last accessed.

Recent frequency is the number of times the page appeared in the previous 50 references.

These features were chosen because they are lightweight and directly related to locality. Recency represents how recently a page was used, while recent frequency captures repeated access during the current workload.

The Decision Tree receives no future reference information. Future knowledge is used only by Optimal when producing training labels and when evaluating the learned decisions.

2.4 Decision Tree Training

A Decision Tree classifier with a maximum depth of five was used. Because each replacement event produces one positive candidate and several negative candidates, class weighting was applied during training.

Quinlan describes Decision Tree learning as induction from classified examples [4]. A particularly relevant observation is that "The essence of induction is to move beyond the training set" [4, p. 87]. For this reason, validation was performed by holding out complete workload traces rather than randomly splitting individual candidate rows. This prevented examples from the same trace from appearing in both the training and validation sets.

The Decision Tree trained on the 16 training traces achieved a candidate-classification accuracy of 82.29% when evaluated on the four held-out traces. With four candidates per replacement event and one positive candidate, approximately 25% of candidate rows are positive. A trivial classifier that always predicts a non-victim would therefore obtain about 75% accuracy. The measured 82.29% is 7.29 percentage points above this trivial accuracy baseline. This candidate-classification metric is different from the Optimal-victim agreement reported later.

For this validation-trained tree, the reported feature importances were approximately 79.84% for recency and 20.16% for recent frequency. These values indicate that the validation tree relied primarily on recency, while recent frequency still contributed additional information. After validation, the model used in the experiments was trained again using the complete generated dataset.

During replacement, the model calculates an eviction score for every resident candidate. The page with the highest predicted eviction probability is selected, with recency used as a tie-breaker if necessary. For each replacement decision, confidence is defined as the predicted eviction probability assigned to the candidate selected by the learned policy. Because the Decision Tree was trained with balanced class weights, these probabilities reflect weighted class proportions within tree leaves and were not

separately calibrated; they are therefore treated here as model confidence scores rather than calibrated probabilities. A serialized copy of the final trained model was also stored using Joblib.

3. Experimental Setup

The primary evaluation was designed as a controlled workload-shift experiment.

- 20 virtual pages
- 4 physical frames
- 2,000 references per evaluation trace
- 1,000 locality-heavy references
- 1,000 random references

During the first phase, approximately 80% of references came from a working set of four pages. The remaining references accessed pages outside that working set. This phase therefore contained temporal structure that could potentially make recency and frequency useful predictors.

At reference 1,000, the workload shifted to uniform random access across all 20 virtual pages. The frame contents were not reset. The second phase therefore continued directly from the state produced by the first phase.

Within each run, FIFO, LRU, Optimal, and the learned policy received exactly the same reference trace. The experiment was repeated across 10 distinct evaluation seeds, from 42 to 51, and the reported performance values are averages across those runs.

The measured quantities were hit ratio, page-fault count, agreement between the learned victim and the Optimal victim, and Decision Tree confidence.

For the victim comparison, Optimal was asked to choose from the same resident candidate set available to the learned policy at that moment. This avoids comparing victim choices from two simulations whose memory states may already have diverged.

The agreement metric records an exact match with the single victim returned by the Optimal implementation. In some memory states, more than one candidate may be equally valid under Optimal. In such a case, an alternative equally good victim could still be recorded as a mismatch. The agreement result should therefore be interpreted as an exact implementation-level match rather than as a complete measure of replacement quality.

4. Results

4.1 Hit Ratio Before and After the Workload Shift

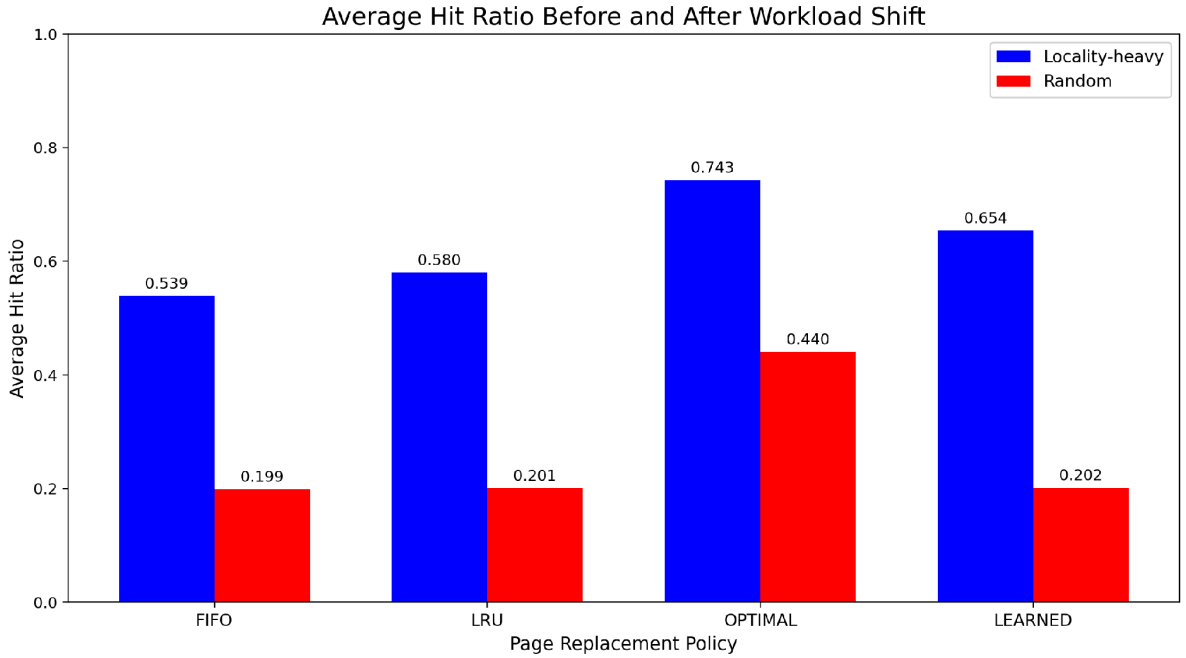


Figure 1. Average Hit Ratio Before and After Workload Shift

Figure 1 shows a clear difference between the two workload phases. During locality-heavy access, the learned policy reached an average hit ratio of 0.6540, exceeding LRU at 0.5802 and FIFO at 0.5389. Optimal remained highest at 0.7431.

After the shift to random access, FIFO, LRU, and the learned policy converged to almost identical hit ratios of 0.1993, 0.2005, and 0.2019, respectively. Optimal remained much higher at 0.4404 because it still had access to future information.

Across the complete traces, the learned policy achieved an overall hit ratio of 0.4280, compared with 0.3904 for LRU and 0.3691 for FIFO.

The locality-phase advantage was consistent across all ten evaluation seeds: the learned policy achieved a higher hit ratio than LRU in every run. Its locality hit ratio ranged from 0.633 to 0.679, compared with 0.545 to 0.603 for LRU.

4.2 Overall Page Faults

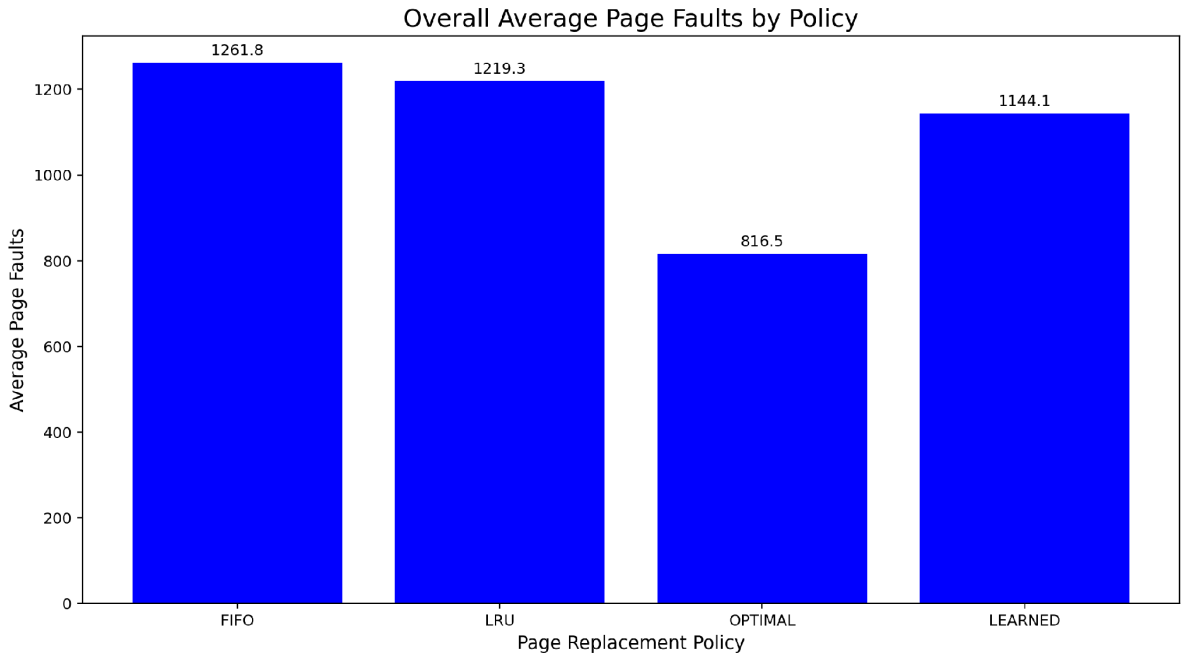


Figure 2. Overall Average Page Faults by Policy

The average page-fault counts were 1261.8 for FIFO, 1219.3 for LRU, 816.5 for Optimal, and 1144.1 for the learned policy.

The learned policy therefore produced approximately 117.7 fewer faults than FIFO and 75.2 fewer faults than LRU. Its advantage was built mainly during the locality-heavy phase and remained large enough to give it the best overall result among the practical policies.

4.3 Confidence and Eviction Correctness

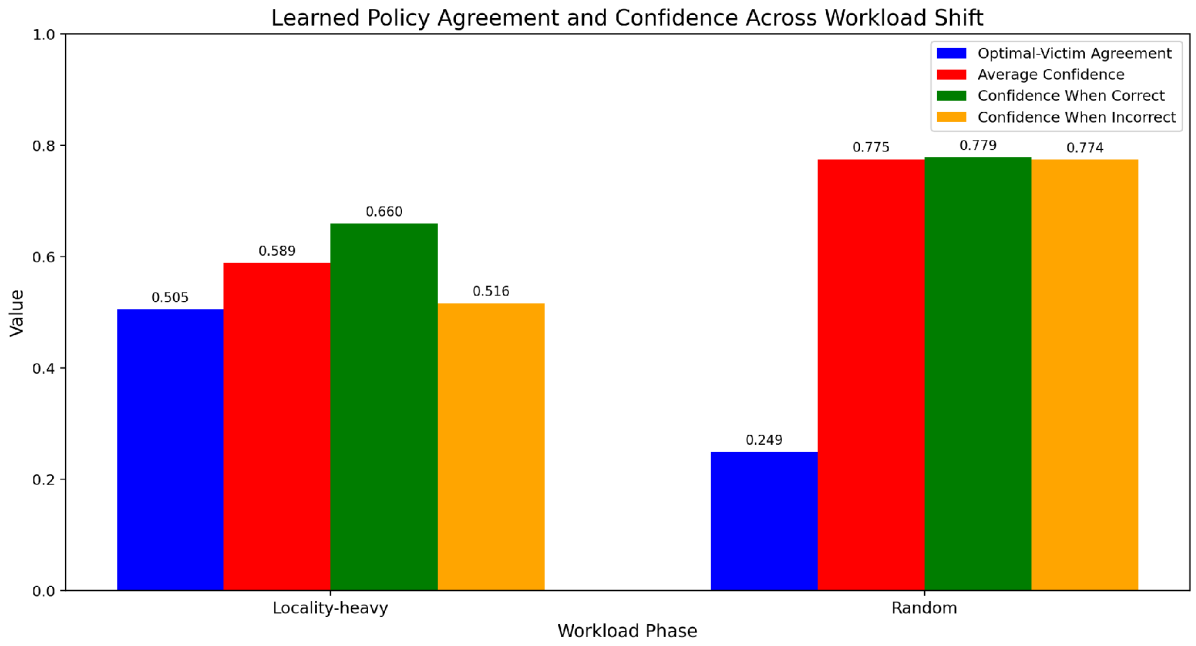


Figure 3. Learned Policy Agreement and Confidence Across Workload Shift

In Figure 3, Optimal-Victim Agreement represents exact agreement between the learned victim and the victim returned by the Optimal implementation.

Metric	Locality-Heavy	Random
Replacement decisions	3420	7981
Correct Optimal-victim matches	1727	1990
Incorrect matches	1693	5991
Optimal-victim agreement	0.5050	0.2493
Average confidence	0.5888	0.7754
Confidence when correct	0.6600	0.7787
Confidence when incorrect	0.5162	0.7742

During locality-heavy access, the learned policy matched the recorded Optimal victim in 50.50% of replacement decisions. Its average confidence was 58.88%, while correct matches had an average confidence of 66.00% compared with 51.62% for incorrect matches.

After the shift to random access, Optimal-victim agreement fell to 24.93%, but average confidence increased to 77.54%. Confidence was almost identical for correct and incorrect matches, at 77.87% and 77.42%, respectively.

The number of replacement decisions also rose substantially, from 3,420 during locality-heavy access to 7,981 during random access.

4.4 Belady Anomaly Test

A secondary experiment used the classic reference string associated with Belady's anomaly: 1 2 3 4 1 2 5 1 2 3 4 5.

Belady, Nelson, and Shedler described the counterintuitive case in which reducing available memory can sometimes improve paging behavior. Their paper states that in anomalous cases "a decrease in the size of the store is accompanied by a decrease in running time" [2, p. 349].

Frames	FIFO Page Faults
3	9
4	10

Increasing the frame count from three to four therefore increased the number of page faults from nine to ten, reproducing the classic anomaly.

Mattson et al. later described stack algorithms, where the pages retained with a smaller memory remain included in those retained with a larger memory [3]. Their work identifies LRU and Optimal as members of this class.

LRU and Optimal showed no anomalous behavior in the test. The learned policy also did not reproduce the anomaly for this reference string and the tested frame counts. However, the learned policy has no proven stack inclusion property. Therefore, this experiment does not establish that it is free from Belady-like anomalous behavior.

5. Analysis of the Workload Shift

The most important finding is the difference between the locality-heavy and random phases.

Under locality-heavy access, recent and repeated use provides useful evidence of near-future reuse, so recency and recent frequency are informative signals. The Decision Tree can combine these signals instead of following one fixed heuristic. This is consistent with the learned policy reaching a hit ratio of 0.6540, compared with 0.5802 for LRU and 0.5389 for FIFO.

The victim-selection result provides additional evidence. With four resident candidates and one recorded Optimal target, a uniform random choice among the candidates would have a 25% probability of matching that recorded target.

The learned policy achieved 50.50% agreement during locality-heavy access, approximately twice that baseline. Historical page behavior therefore contained useful information for the model during this phase.

The behavior changed sharply after the workload became uniformly random. Random workloads had already been included in the training dataset, so the decline cannot be explained simply by saying that the model had never seen random access.

The more important issue is the assumption underlying the input features. Recency and recent frequency both summarize past behavior. Under locality, past behavior provides useful evidence about future reuse. In the random phase used here, page references were generated uniformly across the 20 virtual pages, so recent or frequent use provided far less information about future references. The relationship between the historical features and Optimal's future-based victim choice therefore became weak.

This is reflected in the Optimal-victim agreement, which fell from 50.50% under locality to 24.93% under random access, almost exactly the 25% random-agreement baseline.

The confidence behavior is even more striking. Under locality, correct victim matches had an average confidence of 66.00%, compared with 51.62% for incorrect matches. Confidence therefore carried useful information about victim agreement.

After the workload shift, this distinction disappeared. Correct matches averaged 77.87% confidence, while incorrect matches averaged 77.42%. At the same time, victim agreement dropped sharply while average confidence rose from 58.88% to 77.54%.

In other words, the learned policy became more confident while becoming less correct according to the agreement metric.

This provides strong evidence of overconfident behavior under workload shift. A Decision Tree can still encounter recency and frequency combinations that resemble patterns present in the training set and assign a high eviction probability to them. Under random access, however, those historical patterns contain far less information about future reuse. High classifier confidence therefore does not necessarily indicate a high probability of matching the future-aware Optimal decision.

Optimal's performance reinforces this interpretation. During the random phase, Optimal still achieved a hit ratio of 0.4404, while FIFO, LRU, and the learned policy all remained close to 0.20. Better future-dependent decisions still existed, but the practical policies could not infer them reliably from the limited historical information available.

There are also limitations to the experiment. The workload traces are synthetic rather than collected from real applications. The learned model uses only two features and one Decision Tree configuration, while the main experiment uses a fixed configuration of four frames and twenty virtual pages. In addition, the Decision Tree confidence values were not explicitly calibrated.

Another limitation comes from the training procedure itself. Training candidates were collected from frame states produced while Optimal controlled the replacement process. During deployment, once the learned policy makes a different eviction decision, it may enter frame states that the Optimal training trajectory would never have visited. This difference between teacher-generated training states and learned-policy deployment states may allow errors to accumulate.

The exact Optimal-victim agreement metric also has a limitation. If more than one resident page is equally optimal at a particular step, the implementation records only one of them as the target. A learned choice of another equally optimal victim would therefore count as an incorrect match even though it may be equally good in terms of future faults.

The results should therefore be interpreted as observations from a controlled experiment rather than as evidence that the learned policy universally outperforms classical page-replacement algorithms.

6. Conclusion

This work investigated a lightweight Decision Tree based page-replacement policy under a deliberate workload shift. The model was trained using synthetic locality-heavy, sequential, random, and bursty traces, with Optimal replacement providing the training labels. The learned policy itself used only recency and recent access frequency.

Under locality-heavy access, the learned policy achieved a hit ratio of 0.6540, exceeding both FIFO and LRU. It also produced fewer overall page faults than either practical baseline. Its agreement with the recorded Optimal victim reached 50.50%, substantially above the 25% random-agreement baseline.

After the workload shifted to uniform random access, the learned policy's hit ratio fell to 0.2019, while Optimal-victim agreement dropped to 24.93%. At the same time, average confidence increased to 77.54%, and confidence no longer meaningfully separated correct from incorrect victim matches.

The results therefore show that, within this controlled setup, a lightweight learned policy can improve page replacement when historical access behavior contains exploitable structure. They also show that the same policy can become unreliable and exhibit overconfident behavior when the workload changes and the assumptions behind its features no longer hold.

The Belady test reproduced the classic FIFO anomaly, with page faults increasing from nine to ten when the frame count rose from three to four. The learned policy did not show this behavior on the tested trace, but no general claim of anomaly freedom can be made because the learned policy is not proven to satisfy the stack inclusion property.

Overall, the experiment suggests that learning can strengthen traditional operating-system heuristics under suitable workload conditions, but robustness under workload shift is just as important as performance under favorable conditions.

References

[1] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Pearson, 2015, pp. 209-212.
[2] L. A. Belady, R. A. Nelson, and G. S. Shedler, "An anomaly in space-time characteristics of certain programs running in a paging machine," Communications of the ACM, vol. 12, no. 6, pp. 349-353, 1969. doi: 10.1145/363011.363155.
[3] R. L. Mattson, J. Gecsei, D. R. Slutz, and I. L. Traiger, "Evaluation techniques for storage hierarchies," IBM Systems Journal, vol. 9, no. 2, pp. 78-117, 1970. doi: 10.1147/sj.92.0078.
[4] J. R. Quinlan, "Induction of decision trees," Machine Learning, vol. 1, no. 1, pp. 81-106, 1986. doi: 10.1007/BF00116251.